# Claude Code

*Architecture of an Agentic System*

Mohammad Faiz

2026

Copyright © 2026 Mohammad Faiz. All Rights Reserved.

**No Permission to Use:** No person or entity is permitted to use, copy, modify, merge, publish, distribute, sublicense, or sell any part of this content for any purpose (commercial or non-commercial).

**Personal Use Only:** Only the original author Mohammad Faiz has the authority to use this content in any project or application.

**Attribution:** If you discuss or reference the concepts, you must provide full credit to the original author Mohammad Faiz.

# Table of Contents

**Chapter 1: The Architecture at a Glance**

**Chapter 2: Bootstrap and Initialization**

**Chapter 3: State Management**

**Chapter 4: The API Layer**

**Chapter 5: The Agent Loop**

**Chapter 6: Tools -- From Definition to Execution**

**Chapter 7: Concurrent Tool Execution**

**Chapter 8: Spawning Sub-Agents**

**Chapter 9: Fork Agents and the Prompt Cache**

**Chapter 10: Tasks, Coordination, and Swarms**

**Chapter 11: Memory -- Learning Across Conversations**

**Chapter 12: Extensibility -- Skills and Hooks**

**Chapter 13: The Terminal UI**

**Chapter 14: Input and Interaction**

**Chapter 15: MCP -- The Universal Tool Protocol**

**Chapter 16: Remote Control and Cloud Execution**

**Chapter 17: Performance -- Every Millisecond and Token Counts**

**Chapter 18: What We Learned**

# Chapter 1: The Architecture at a Glance

## The Core Abstractions

Claude Code is built around six core abstractions that work together to create an agentic system. Understanding these six concepts provides the foundation for understanding everything else.

**The Agent Loop** is the heart of the system. It is a `while(true)` cycle that streams responses from a language model, extracts tool calls, executes them, and feeds results back into the conversation. The loop handles ten terminal states and seven continuation conditions, each with specific semantics for when and why the loop should exit or continue.

**Tools** are the actions the agent can take. Each tool is a self-contained unit with a name, description, input schema, and execution logic. Tools declare their own safety properties: whether they are read-only, whether they can run concurrently, what permission mode they require. The tool system spans 40+ built-in implementations and an extensible registry that can incorporate MCP servers.

**State Management** handles the conversation history, the application state, and the various caches that make the system responsive. The message history is immutable -- every turn appends new messages rather than modifying existing ones. This immutability is what enables features like rewind, resume, and fork-based sub-agents.

**The Rendering Engine** translates the internal state into terminal output. It is a custom React-based renderer optimized for 60fps streaming output, with packed typed arrays, pool-based string interning, and cell-level diffing. The renderer is what makes the terminal feel like a modern interface rather than a scrolling log.

**Sub-Agents** enable parallel work. When the model encounters a task that benefits from delegation, it spawns a child agent with its own conversation loop, tool set, and permission boundary. Fork agents share the parent's prompt cache, making parallel delegation economical. Coordinator mode and swarm teams provide structured multi-agent orchestration.

**Memory** persists knowledge across sessions. The memory system uses Markdown files on disk, organized by a four-type taxonomy (user, feedback, project, reference), with LLM-powered recall that selects relevant memories on demand. The file-based approach makes the agent's knowledge transparent and human-editable.

## System Overview

```
┌──────────────────────────────────────────────────────────────────┐
│                        SYSTEM OVERVIEW                           │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   ┌──────────────┐      ┌──────────────┐      ┌──────────────┐   │
│   │  Terminal UI │◄────►│  Core Engine │◄────►│  Tool System │   │
│   │              │      │              │      │              │   │
│   │ • Input      │      │ • Agent Loop │      │ • 40+ Tools  │   │
│   │ • Display    │      │ • State Mgr  │      │ • Registry   │   │
│   │ • Rendering  │      │ • Renderer   │      │ • Permissions│   │
│   └──────────────┘      └──────────────┘      └──────────────┘   │
│          ▲                    ▲                    ▲             │
│          │                    │                    │             │
│          └────────────────────┼────────────────────┘             │
│                               │                                  │
│                     ┌─────────┴──────────┐                       │
│                     │      External      │                       │
│                     │  • Anthropic API   │                       │
│                     │  • Filesystem      │                       │
│                     │  • MCP Servers     │                       │
│                     └────────────────────┘                       │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

## The Data Flow

A single turn through Claude Code follows this path:

1. The user submits a prompt, which is normalized and appended to the message history.
2. The system assembles the context: system prompt, tool definitions, conversation history, and any recalled memories.
3. The API is called with streaming enabled. Tokens arrive one at a time.
4. The streaming parser extracts content blocks: text for the user, thinking blocks for reasoning, and tool_use blocks for actions.
5. When a complete tool_use block arrives, the tool system validates the input, checks permissions, and executes.
6. Tool results are serialized and appended to the conversation as tool_result blocks.
7. The loop continues until the model signals completion or a terminal state is reached.

This simple cycle -- stream, collect, execute, append, repeat -- is the foundation. Everything else is optimization and edge case handling.

## Design Philosophy

Three principles guide the architecture:

**Explicit over implicit.** The agent loop is a single async generator function, not a distributed callback system. Every state transition is visible in the control flow. Every terminal state is encoded in a discriminated union. When something goes wrong, there is one place to look.

**Fail closed.** Tools default to serial execution until proven safe for concurrency. Permissions default to prompting until explicitly allowed. New features are gated behind feature flags that default to off. The system errs on the side of safety.

**Measure everything.** Startup has 50+ profiling checkpoints. API calls track token usage and cache hit rates. The rendering engine reports frame timing. Performance optimizations are driven by data, not intuition.

## What This Book Covers

The chapters that follow examine each subsystem in depth. Chapter 2 covers bootstrap -- how the system initializes from a cold start. Chapter 3 examines state management in detail. Chapter 4 traces the API layer. Chapter 5 is the heart of the book: the agent loop itself. Chapters 6-7 cover the tool system and concurrent execution. Chapters 8-10 examine sub-agents, fork agents, and coordination patterns. Chapters 11-12 cover memory and extensibility. Chapters 13-14 dive into the terminal UI and input handling. Chapters 15-16 examine external protocols: MCP and remote execution. Chapter 17 covers performance optimization. Chapter 18 summarizes the architectural lessons.

The source code is the ultimate reference. This book explains the why and how of the design. The code shows the what. Both are necessary for a complete understanding.

# Chapter 2: Bootstrap and Initialization

## The Entry Point

Every Claude Code session begins with `main.tsx`. The file is deceptively small -- approximately 200 lines -- but it orchestrates a complex initialization sequence that brings the system from cold start to interactive REPL in under two seconds.

The entry point performs three distinct phases of work:

**Phase 1: Pre-initialization.** Before any React code runs, the system performs module-level I/O that can happen in parallel with module loading. Two macOS keychain reads fire as fire-and-forget promises. An API preconnection warms the TCP+TLS handshake. These operations would cost ~65ms if done sequentially; by launching them at module scope, they overlap with ~135ms of JavaScript module loading.

**Phase 2: CLI parsing.** The command-line interface supports multiple entry modes: interactive REPL, one-shot command execution, MCP server management, and daemon mode. The parser uses `commander` to extract flags, validate arguments, and dispatch to the appropriate handler.

**Phase 3: REPL initialization.** For interactive mode, the system creates an Ink instance (the custom React renderer for terminals), mounts the REPL component, and begins the first query loop. This phase is where most of the complexity lives.

## The Bootstrap State Machine

```
┌──────────────────────────────────────────────────────────────────┐
│                     BOOTSTRAP STATE MACHINE                      │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│      ┌──────────────┐                                            │
│      │     [*]      │                                            │
│      └──────────────┘                                            │
│             │                                                    │
│             ▼                                                    │
│      ┌──────────────┐        resume existing      ┌──────────────┐
│      │ initializing │──────────────────────────▶│loading_history│
│      └──────────────┘                             └──────────────┘
│             │                                          │
│             │ start fresh session                      │ history
│             │                                          │ loaded
│             ▼                                          ▼
│      ┌──────────────┐                            ┌──────────────┐
│      │creating_new  │                            │    ready     │──────────▶ exit
│      └──────────────┘                            └──────────────┘
│                                                          │
│                                                          │ first input
│                                                          ▼
│                                                   ┌──────────────┐
│                                                   │  querying    │
│                                                   └──────────────┘
│                                                          │
│                                                          │ API call
│                                                          ▼
│                                                   ┌──────────────┐
│                                                   │  streaming   │──────────▶ ready
│                                                   └──────────────┘
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

The `initializing` state is brief -- typically under 100ms -- but it handles critical setup: loading the system prompt, discovering skills and agents, initializing MCP connections, and restoring session state if resuming.

## Session Restoration

When resuming a session, Claude Code must reconstruct the conversation history, the application state, and the various caches that were lost when the process exited. This happens through a layered restoration process:

**Transcript loading.** The main conversation is stored as a JSONL file -- one JSON object per line, each representing a message. The loader streams this file, validates each message against the schema, and appends valid messages to the conversation history. Corrupted lines are skipped with a warning.

**Sidechain transcripts.** Each sub-agent has its own transcript file. These are loaded lazily -- only when the parent agent references the sub-agent's results. The sidechain format is identical to the main transcript, enabling the same validation and loading code to handle both.

**State hydration.** Application state -- pending tasks, permission modes, file caches -- is serialized to disk on exit and restored on resume. The serialization uses a selective approach: only state that cannot be reconstructed is persisted. File caches, for example, are not persisted -- they are rebuilt on demand.

## Deferred Loading

Not everything loads at startup. Heavy subsystems are deferred until first use:

- **OpenTelemetry** (~400KB + ~700KB gRPC) loads only when telemetry is enabled.
- **Syntax highlighting** loads via React Suspense when the first code block appears.
- **MCP servers** connect in batches after the REPL is interactive.
- **Skills** load frontmatter at startup but full content only on invocation.

The deferral strategy keeps startup fast while ensuring all functionality is available when needed. The user sees an interactive prompt in under two seconds; heavy initialization happens in the background.

## Fast-Path Dispatch

Some CLI commands never need the full REPL. The `claude mcp` command manages MCP server configurations without mounting the React UI. The `claude daemon` command starts a headless server. These fast paths skip the entire REPL initialization, reducing startup time to under 500ms.

The dispatch logic lives at the very top of `main.tsx`, before any heavy imports:

```
if (process.argv.includes('mcp')) {
  // Fast path: MCP management
  await runMcpCommand();
  return;
}

if (process.argv.includes('daemon')) {
  // Fast path: headless server
  await runDaemon();
  return;
}

// Full REPL initialization
const { REPL } = await import('./REPL');
```

The dynamic `import()` ensures that the REPL code -- and all its dependencies -- are only loaded when needed. A simple `claude --version` never pays the cost of React, Ink, or the tool system.

## Startup Profiling

Claude Code ships with 50+ startup profiling checkpoints. Each checkpoint records the elapsed time since the previous checkpoint, enabling detailed analysis of where startup time is spent.

The checkpoints are sampled at 100% for internal users and 0.5% for external users. This produces statistically significant data without overwhelming the telemetry pipeline. The data drives optimization decisions: when a checkpoint shows 65ms spent on sequential keychain reads, the fix is to parallelize them.

## Error Handling During Bootstrap

Errors during initialization are handled differently than runtime errors. A missing system prompt file is fatal -- the system cannot function without it. A corrupted transcript line is non-fatal -- the system skips the line and continues loading.

The error handling strategy is:

- **Fatal errors** exit with a clear message and non-zero status code.
- **Recoverable errors** log a warning and continue with degraded functionality.
- **Silent failures** are avoided -- every error is either fixed or explained.

The bootstrap phase is the most fragile part of the system. Once the REPL is running, the agent loop's error handling provides multiple recovery paths. During bootstrap, there is no loop yet -- errors must be handled explicitly at each step.

## Apply This: Designing Fast Startup

**Parallelize I/O with module loading.** CPU-bound work (module loading) and I/O-bound work (keychain reads, network handshakes) can happen in parallel. Launch I/O before imports complete.

**Use fast-path dispatch.** Not every command needs the full application. Simple commands should exit before heavy subsystems load.

**Defer until first use.** Heavy subsystems should load on demand, not at startup. The user sees an interactive prompt faster; background work fills in the gaps.

**Measure everything.** Profiling checkpoints reveal where time is actually spent. Optimization without measurement is guesswork.

The bootstrap phase sets the tone for the entire user experience. A two-second startup feels instant. A five-second startup feels slow. The difference is not in the total work done -- it is in when the work happens and what the user sees while waiting.

# Chapter 3: State Management

## The Immutable Message History

At the center of Claude Code's state management is the message history: an array of messages that grows with each turn of the conversation. This array is immutable -- messages are never modified after creation, only appended.

The immutability is not a performance optimization. It is a correctness requirement. When a sub-agent forks from a parent, it receives a copy of the parent's message history. If the parent could modify messages after the fork, the child's history would diverge from reality. Immutability guarantees that every agent sees a consistent, append-only log of the conversation.

The message types form a discriminated union:

```
type Message =
  | UserMessage
  | AssistantMessage
  | ToolUseBlock
  | ToolResultBlock
  | SystemMessage
  | AttachmentMessage
```

Each message has a unique ID, a timestamp, and content appropriate to its type. The `ToolUseBlock` and `ToolResultBlock` are linked by a shared `toolUseId`, enabling the system to match results with their originating calls.

## The App State Store

Beyond the message history, Claude Code maintains an application state store using React's `useState` and `useReducer` patterns. The store is organized into domains:

- **messages**: The conversation history (immutable array).
- **tasks**: Background task states (agents, shell commands, workflows).
- **permissions**: Current permission mode and pending requests.
- **ui**: UI state (scroll position, selection, search query).

- **config**: User settings and feature flags.

The store is accessed through a context provider that wraps the REPL component. Child components receive the state and dispatch functions via React context, enabling reactive updates throughout the component tree.

## State Persistence

Not all state is persisted to disk. The persistence strategy is selective:

| State | Persisted? | Rationale |
|---|---|---|
| Message history | Yes | Required for resume |
| Task states | Partial | Running tasks become "unknown" on resume |
| File caches | No | Rebuilt on demand |
| UI state | No | Fresh start is cleaner |
| Permission modes | Yes | User's security preference |

The persistence format is JSONL for message histories (append-only, corruption-resistant) and JSON for application state (structured, schema-validated). On exit, the system writes both files atomically: first to a temporary file, then a rename operation that replaces the old file.

## The Read File State Cache

One of the most heavily used caches is the `readFileState`: an LRU cache of file contents and timestamps. When the model requests a file, the system first checks this cache. A hit avoids a disk read; a miss triggers a read and caches the result.

The cache has two important properties:

**Size-limited.** The cache has a maximum size (configurable, default 50MB). When the limit is exceeded, the least-recently-used entries are evicted. This prevents unbounded memory growth in long sessions.

**Staleness-aware.** Each cache entry stores the file's modification time at the moment of caching. Before returning a cached result, the system checks if the file has been modified on disk. If so, the cache entry is invalidated and the file is re-read.

The staleness check is what enables safe caching in a development environment where files change frequently. Without it, the model would see stale file contents and make incorrect decisions.

## Content Replacement State

A subtle but important piece of state is the `ContentReplacementState`. This tracks tool results that have been persisted to disk because they exceeded the per-tool size limit. Instead of including the full result in the conversation, the system stores it in `~/.claude/tool-results/{hash}.txt` and includes a `<persisted-output>` wrapper with a preview and file path.

The `ContentReplacementState` tracks:

- Which tool results have been persisted.
- The file path for each persisted result.
- The aggregate size of all persisted results (to enforce per-conversation limits).

This state is critical for cache stability. When a fork agent is created, it receives a clone of the parent's `ContentReplacementState`. This ensures that if the child reads a persisted result, it knows where to find it -- even though the child's conversation history only contains the `<persisted-output>` wrapper, not the full content.

## State Isolation for Sub-Agents

When a sub-agent spawns, it receives a carefully constructed subset of the parent's state:

- **Messages**: Cloned (for fork agents) or fresh (for normal agents).
- **File cache**: Cloned (shallow copy, shared content strings).
- **Abort controller**: Independent for async agents, shared for sync agents.
- **App state**: Isolated for async agents, shared for sync agents.

The isolation decisions are deliberate. An async agent should not see the parent's UI state changes -- that would cause the parent's interface to jump unexpectedly. But an async agent must still be able to update task state (progress, completion) that the parent observes. The split channel -- isolated `setAppState` but shared `setAppStateForTasks` -- solves both requirements.

## Apply This: Designing Agent State

**Make message history immutable.** Append-only semantics enable fork agents, resume, and rewind. The cost (copying arrays) is negligible compared to the benefits.

**Use LRU caches for file content.** Disk reads are slow; memory is fast. A bounded cache with staleness checks gives the performance of caching with the correctness of fresh reads.

**Be selective about persistence.** Not all state needs to survive process restart. Persist what matters; rebuild what can be reconstructed.

**Isolate sub-agent state carefully.** Shared state enables coordination; isolated state prevents interference. Choose the right boundary for each concern.

# Chapter 4: The API Layer

## Streaming vs. Non-Streaming

Claude Code uses the Anthropic API's raw streaming interface rather than the SDK's helper classes. The difference matters for performance: the SDK's `BetaMessageStream` calls `partialParse()` on every `input_json_delta`, which is O(n^2) in tool input length. Claude Code accumulates raw strings and parses once when the block is complete.

The streaming implementation handles three content block types:

- **text**: Plain text output for the user.
- **thinking**: Extended reasoning (when enabled via the thinking feature).
- **tool_use**: A request to execute a tool, with name and input JSON.

Each block arrives as a series of deltas. The streaming parser accumulates these deltas into complete blocks, yielding them to the agent loop as they finish.

## The Streaming Parser

```
┌──────────────────────────────────────────────────────────────┐
│                    STREAMING PARSER STATE MACHINE            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   ┌──────────┐        SSE event line      ┌──────────────┐   │
│   │  idle    │──────────────────────────▶│ reading_event│   │
│   └──────────┘                            └──────────────┘   │
│        │                                        │            │
│        │                                        │ data: line │
│        │                                        ▼            │
│        │                              ┌──────────────┐       │
│        │                              │ reading_data │◀── more data │
│        │                              └──────────────┘       │
│        │                                        │            │
│        │                                        │ empty line │
│        │                                        ▼            │
│        │                              ┌──────────────┐       │
│        │                              │  processing  │       │
│        │                              └──────────────┘       │
│        │                                        │            │
│        │          block incomplete              │ block complete │
│        └────────────────────────────────────────┴──────────▶ yielding │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

The parser maintains partial state for each active block. A `tool_use` block might span dozens of SSE events; the parser accumulates the name and input JSON incrementally, only yielding when the block is marked complete.

## API Preconnection

To reduce latency on the first API call, Claude Code implements API preconnection. During initialization, it fires a `HEAD` request to the Anthropic API endpoint. This warms the TCP connection and TLS handshake, overlapping the ~100-200ms network setup with other initialization work.

The preconnection fires after transport configuration is complete (`applyExtraCACertsFromConfig()` and `configureGlobalAgents()`) so the warmed connection uses the correct settings. In interactive mode, the overlap is unbounded -- the connection warms while the user types their first prompt.

## Error Handling and Retries

API calls can fail for many reasons: network errors, rate limits, server errors, authentication failures. The error handling strategy is layered:

**Retryable errors** (network timeouts, 5xx server errors) trigger automatic retry with exponential backoff. The retry policy is: up to 3 attempts, with delays of 1s, 2s, and 4s.

**Non-retryable errors** (4xx client errors, authentication failures) fail immediately with a clear error message. These indicate a problem that retrying will not fix.

**Streaming failures** mid-response are handled specially. If the stream drops after partial content has been received, the system attempts to resume from the last complete message. This requires server-side support for the `resuming` parameter, which Claude Code uses when available.

## The Streaming Watchdog

A streaming watchdog (`CLAUDE_STREAM_IDLE_TIMEOUT_MS`, default 90 seconds) aborts and retries if no chunks arrive within the timeout. This handles cases where the connection is silently dropped -- the TCP connection remains open, but no data flows.

On watchdog timeout, the system:

1. Aborts the current request.
2. Attempts to retry with the same parameters.
3. If retry fails, falls back to non-streaming `messages.create()`.

The fallback to non-streaming is a last resort. It sacrifices the responsiveness of streaming for the reliability of a simple request/response.

## Token Usage Tracking

Every API response includes token usage statistics. Claude Code tracks these across the conversation:

- **Input tokens**: Cumulative (the full conversation is re-sent each turn).
- **Output tokens**: Per-turn (new tokens generated each turn).
- **Cache hits**: Tokens that hit the prompt cache (90% discount).

The tracking is accurate because it uses the API's actual `usage` field, not client-side estimation. This accounts for prompt caching credits, thinking tokens, and server-side transformations that would be difficult to replicate client-side.

## Prompt Cache Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                     PROMPT CACHE STRUCTURE                       │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │  CLI identity, tool instructions, code style rules       │   │
│   │  ═══════════════════════════════════════════════════      │   │
│   │  GLOBALLY CACHEABLE (90% discount)                       │   │
│   └──────────────────────────────────────────────────────────┘   │
│                              │                                   │
│                              ▼                                   │
│                     __DYNAMIC_BOUNDARY__                         │
│                              │                                   │
│                              ▼                                   │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │  Date, memory files, CLAUDE.md, output preferences       │   │
│   │  ═══════════════════════════════════════════════════      │   │
│   │  PER-SESSION (full price)                                │   │
│   └──────────────────────────────────────────────────────────┘   │
│                              │                                   │
│                              ▼                                   │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │  Conversation history (grows each turn)                  │   │
│   │  ═══════════════════════════════════════════════════      │   │
│   │  GROWS EACH TURN (full price)                            │   │
│   └──────────────────────────────────────────────────────────┘   │
│                              │                                   │
│                              ▼                                   │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │  Tool results (volatile)                                 │   │
│   │  ═══════════════════════════════════════════════════      │   │
│   │  VOLATILE (full price)                                   │   │
│   └──────────────────────────────────────────────────────────┘   │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

The `__DYNAMIC_BOUNDARY__` marker separates the globally cacheable prefix (same for all users of the same Claude Code version) from the per-session content. When `shouldUseGlobalCacheScope()` returns true, entries before this boundary get `scope: 'global'`, enabling cross-user cache sharing.

## Apply This: Designing API Layers

**Use raw streaming for performance.** SDK helpers often add overhead. Accumulate raw deltas and parse once at block completion.

**Preconnect to reduce latency.** A HEAD request during initialization overlaps network setup with other work.

**Structure prompts for cache stability.** Stable content first, volatile content last. A single token change mid-prefix invalidates everything after it.

**Track actual usage, not estimates.** The API's usage field accounts for complexities that are hard to replicate client-side.

# Chapter 5: The Agent Loop

## The Heart of the System

The `query()` function in `query.ts` is the heart of Claude Code. It is an async generator that drives the entire agentic interaction: streaming model responses, collecting tool calls, executing them, and feeding results back. The function is approximately 1,700 lines, and every line exists for a reason.

The loop follows a simple pattern that repeats until a terminal state is reached:

```
while (true) {
  // 1. Stream the model's response
  for await (const chunk of streamResponse(messages)) {
    yield { type: 'content', chunk };
  }

  // 2. Extract tool calls from the response
  const toolCalls = extractToolCalls(lastAssistantMessage);

  if (toolCalls.length === 0) {
    // No tools requested -- the model is done
    return { type: 'completed', reason: 'natural_finish' };
  }

  // 3. Execute tool calls
  for (const toolCall of toolCalls) {
    const result = await executeTool(toolCall);
    yield { type: 'tool_result', result };
  }

  // 4. Append results to conversation
  messages.push(...toolResults);

  // 5. Continue to next turn
}
```

This is the foundation. The actual implementation adds error handling, permission checks, concurrent execution, sub-agent spawning, and ten terminal states -- but the core pattern remains: stream, collect, execute, append, repeat.

## Ten Terminal States

The loop can terminate in exactly ten ways, encoded as a discriminated union:

| State | Meaning |
|---|---|
| `completed` | Natural finish -- the model indicated completion |
| `aborted` | User pressed Ctrl+C or Escape |
| `max_turns_reached` | Turn limit exceeded (safety guard) |
| `permission_denied` | User rejected a permission request |
| `tool_error_fatal` | A tool failed with an unrecoverable error |
| `api_error` | The API returned an error |
| `rate_limited` | Rate limit hit, retry exhausted |
| `context_limit` | Context window exceeded |
| `session_ended` | Session was terminated externally |
| `unknown_error` | Unexpected error (catch-all) |

Each terminal state carries context: the message history at termination, the reason for termination, and any error details. This enables the caller to decide what to do next: resume, retry, or report failure.

## Seven Continuation Conditions

Not every tool result should trigger a new API call. The loop checks seven conditions to decide whether to continue:

1. **User interruption**: If the user typed a new message during tool execution, process it before continuing.
2. **Pending permission**: If a permission request is awaiting user response, wait for it.
3. **Sub-agent completion**: If a sub-agent finished, its results may require immediate handling.
4. **Hook continuation**: A Stop hook may force continuation even if the model thinks it is done.
5. **Context compaction**: If the context window is near full, compact before continuing.
6. **Memory injection**: If relevant memories were recalled, inject them before the next turn.
7. **Natural continuation**: None of the above -- proceed to the next API call.

These conditions are checked in order, with earlier conditions taking precedence. A user interruption is always handled before checking for sub-agent completion, ensuring the user feels in control.

## The Generator Pattern

The `query()` function is an async generator, not a regular async function. This design choice enables:

**Streaming.** The caller receives content as it arrives from the API, not all at once at the end. This enables real-time display of the model's output.

**Cancellation.** The caller can stop the generator at any point by calling `return()` on the async iterator. This triggers cleanup in the generator's `finally` block.

**Interleaving.** The caller can process yielded values while the generator is still running. This enables concurrent tool execution and background task management.

The generator yields a discriminated union of events:

```
type QueryEvent =
  | { type: 'content'; chunk: ContentChunk }
  | { type: 'thinking'; chunk: ThinkingChunk }
  | { type: 'tool_use'; toolUse: ToolUseBlock }
  | { type: 'tool_result'; result: ToolResultBlock }
  | { type: 'message_complete'; message: AssistantMessage }
  | { type: 'turn_complete' }
  | { type: 'completed'; result: TerminalState }
```

## Error Recovery

Errors during the loop are handled with a tiered strategy:

**Transient errors** (network timeouts, rate limits) trigger retry with backoff. The loop continues from the same state.

**Recoverable errors** (tool failures, permission denials) are reported to the model as tool results. The model can then decide how to proceed -- retry, try a different approach, or ask the user.

**Fatal errors** (API authentication failure, context limit exceeded) terminate the loop with an appropriate terminal state.

The key insight is that most errors should not kill the loop. A tool failure is information for the model, not a reason to crash. The model is often better equipped to recover than the code that detected the error.

## Apply This: Designing Agent Loops

**Use a generator for the core loop.** Streaming, cancellation, and interleaving are natural with generators. A callback-based architecture scatters the logic across handlers.

**Encode terminal states explicitly.** A discriminated union makes every exit path visible. The type system enforces that all states are handled.

**Let the model recover from errors.** Report tool failures as results, not exceptions. The model's reasoning ability is often the best recovery mechanism.

**Check continuation conditions in priority order.** User input should always take precedence over background tasks. The order defines the user experience.

# Chapter 6: Tools -- From Definition to Execution

## The Nervous System

Chapter 5 showed you the agent loop -- the `while(true)` that streams model responses, collects tool calls, and feeds results back. The loop is the heartbeat. But the heartbeat is meaningless without the nervous system that translates "the model wants to run `git status`" into an actual shell command, with permission checks, result budgeting, and error handling.

The tool system is that nervous system. It spans 40+ tool implementations, a centralized registry with feature-flag gating, a 14-step execution pipeline, a permission resolver with seven modes, and a streaming executor that starts tools before the model finishes its response.

Every tool call in Claude Code -- every file read, every shell command, every grep, every sub-agent dispatch -- flows through the same pipeline. The uniformity is the point: whether the tool is a built-in Bash executor or a third-party MCP server, it gets the same validation, the same permission checks, the same result budgeting, the same error classification.

The `Tool` interface has approximately 45 members. That sounds overwhelming, but only five matter for understanding how the system works:

1. **`call()`** -- execute the tool
2. **`inputSchema`** -- validate and parse the input
3. **`isConcurrencySafe()`** -- can this run in parallel?
4. **`checkPermissions()`** -- is this allowed?
5. **`validateInput()`** -- does this input make semantic sense?

Everything else -- the 12 rendering methods, the analytics hooks, the search hints -- exists to support the UI and telemetry layers. Start with the five, and the rest falls into place.

## The 14-Step Execution Pipeline

```
┌──────────────────────────────────────────────────────────────────┐
│                 14-STEP TOOL EXECUTION PIPELINE                  │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  1. Tool Lookup ──────────────────────────────────────────────▶  │
│  2. Abort Check ──────────────────────────────────────────────▶  │
│  3. Zod Validation ──────────▶ [Fails] ──▶ Input validation err  │
│       │                        [Passes]                                  │
│       ▼                                                          │
│  4. Semantic Validation ─────▶ [Fails] ──▶ Tool-specific error   │
│       │                        [Passes]                                  │
│       ▼                                                          │
│  5. Speculative Classifier Start ─────────────────────────────▶  │
│  6. Input Backfill (clone, not mutate) ───────────────────────▶  │
│  7. PreToolUse Hooks ────────▶ [Denies] ──▶ Hook rejection      |
│       │                        [Stops] ─────▶ Abort execution    │
│       │                        [Passes]                                  │
│       ▼                                                          │
│  8. Permission Resolution ────────────────────────────────────▶  │
│  9. Permission Denied? ──────▶ [Yes] ────▶ Permission denied     │
│       │                        [No]                                      │
│       ▼                                                          │
│  10. Tool Execution ──────────────────────────────────────────▶  │
│  11. Result Budgeting ────────────────────────────────────────▶  │
│  12. PostToolUse Hooks ───────────────────────────────────────▶  │
│  13. New Messages ────────────────────────────────────────────▶  │
│  14. Error Handling ──────────────────────────────────────────▶  │
│                                                                  │
│   ┌──────────────────────────────────────────────────────┐       │
│   │  Tool Result ──▶ Conversation History                 |       │
│   └──────────────────────────────────────────────────────┘       │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

### Steps 1-4: Validation

**Tool Lookup** falls back to `getAllBaseTools()` for alias matches, handling transcripts from older sessions where a tool was renamed. **Abort Check** prevents wasted computation on tool calls queued before Ctrl+C propagated. **Zod Validation** catches type mismatches; for deferred tools, the error appends a hint to call ToolSearch first. **Semantic Validation** goes beyond schema conformance -- FileEditTool rejects no-op edits, BashTool blocks standalone `sleep` when MonitorTool is available.

### Steps 5-6: Preparation

**Speculative Classifier Start** kicks off the auto-mode security classifier in parallel for Bash commands, shaving hundreds of milliseconds off the common path. **Input Backfill** clones the parsed input and adds derived fields (expanding `~/foo.txt` to absolute paths) for hooks and permissions, preserving the original for transcript stability.

### Steps 7-9: Permission

**PreToolUse Hooks** are the extension mechanism -- they can make permission decisions, modify inputs, inject context, or stop execution entirely. **Permission Resolution** bridges hooks and the general permission system: if a hook already decided, that is final; otherwise `canUseTool()` triggers rule matching, tool-specific checks, mode-based defaults, and interactive prompts. **Permission Denied Handling** builds an error message and executes `PermissionDenied` hooks.

### Steps 10-14: Execution and Cleanup

**Tool Execution** runs the actual `call()` with the original input. **Result Budgeting** persists oversized output to `~/.claude/tool-results/{hash}.txt` and replaces it with a preview. **PostToolUse Hooks** can modify MCP output or block continuation. **New Messages** are appended (sub-agent transcripts, system reminders). **Error Handling** classifies errors for telemetry, extracts safe strings from potentially mangled names, and emits OTel events.

# The Permission System

## Seven Modes

| Mode | Behavior |
|---|---|
| `default` | Tool-specific checks; prompt user for unrecognized operations |
| `acceptEdits` | Auto-allow file edits; prompt for other operations |
| `plan` | Read-only -- deny all write operations |
| `dontAsk` | Auto-deny anything that would normally prompt (background agents) |
| `bypassPermissions` | Allow everything without prompting |
| `auto` | Use the transcript classifier to decide (feature-flagged) |
| `bubble` | Internal mode for sub-agents that escalate to the parent |

# Result Budgeting

## Per-Tool Size Limits

Each tool declares `maxResultSizeChars`:

| Tool | maxResultSizeChars | Rationale |
|---|---|---|
| BashTool | 30,000 | Enough for most useful output |
| FileEditTool | 100,000 | Diffs can be large but the model needs them |
| GrepTool | 100,000 | Search results with context lines add up fast |
| FileReadTool | Infinity | Self-bounds via its own token limits; persisting would create circular Read loops |

When a result exceeds the threshold, the full content is saved to disk and replaced with a `<persisted-output>` wrapper containing a preview and file path. The model can then use `Read` to access the full output if needed.

## Apply This: Designing a Tool System

**Fail-closed defaults.** New tools should be conservative until explicitly marked otherwise. A developer who forgets to set a flag gets the safe behavior, not the dangerous one.

**Input-dependent safety.** `isConcurrencySafe(input)` and `isReadOnly(input)` take the parsed input because the same tool at different inputs has different safety profiles.

**Layer your permissions.** Tool-specific checks, rule-based matching, mode-based defaults, interactive prompts, and automated classifiers each handle different cases. No single mechanism is sufficient.

**Budget results, not just inputs.** Token limits on input are standard. But tool results can be arbitrarily large and they accumulate across turns. Per-tool limits prevent individual explosions. Aggregate conversation limits prevent cumulative overflow.

# Chapter 7: Concurrent Tool Execution

## The Cost of Waiting

Chapter 6 traced the lifecycle of a single tool call -- from the raw `tool_use` block in the API response through input validation, permission checks, execution, and result formatting. That pipeline handles one tool. But the model rarely requests just one.

A typical Claude Code interaction involves three to five tool calls per turn. "Read these two files, grep for this pattern, then edit this function." The model emits all of those in a single response. If each tool takes 200 milliseconds, running them sequentially costs a full second. If the Read and Grep calls are independent -- and they are -- running them in parallel cuts that to 200 milliseconds. Five-to-one improvement, free.

But not all tools are independent. An Edit that modifies `config.ts` cannot run concurrently with another Edit that modifies `config.ts`. A Bash command that creates a directory must complete before a Bash command that writes a file into that directory. Concurrency is not a global property of a tool. It is a property of a specific tool invocation with specific inputs.

This is the insight that drives the entire concurrency system: **safety is per-call, not per-tool-type**. `Bash("ls -la")` is safe to parallelize. `Bash("rm -rf build/")` is not. The same tool, different inputs, different concurrency classification. The system must inspect the input before deciding.

## The Partition Algorithm

The entry point is `partitionToolCalls()` in `toolOrchestration.ts`. It takes an ordered array of `ToolUseBlock` messages and produces an array of batches, where each batch is either "all concurrent-safe" or "a single serial tool."

```
// Pseudocode -- illustrates the partition algorithm
type Group = { parallel: boolean; calls: ToolCall[] }

function groupBySafety(calls: ToolCall[], registry: ToolRegistry): Group[] {
  return calls.reduce((groups, call) => {
    const def = registry.lookup(call.name)
    const input = def?.schema.safeParse(call.input)
    // Fail-closed: parse failure or exception -> serial
    const safe = input?.success
      ? tryCatch(() => def.isParallelSafe(input.data), false)
      : false
    // Merge consecutive safe calls into one group
    if (safe && groups.at(-1)?.parallel) {
      groups.at(-1)!.calls.push(call)
    } else {
      groups.push({ parallel: safe, calls: [call] })
    }
    return groups
  }, [] as Group[])
}
```

The algorithm walks the array left to right. For each tool call:

1. **Look up the tool definition** by name.
2. **Parse the input** with the tool's Zod schema via `safeParse()`. If parsing fails, the tool is conservatively classified as not concurrency-safe.
3. **Call `isConcurrencySafe(parsedInput)`** on the tool definition. This is where per-input classification happens.
4. **Merge or create a batch.** If the current tool is concurrency-safe AND the most recent batch is also concurrency-safe, append to that batch. Otherwise, start a new batch.

# Batch Execution

## Concurrent Batches

For a concurrent batch, `runToolsConcurrently()` fires all tools in parallel using an `all()` utility that caps active generators at the concurrency limit:

```
// Pseudocode -- illustrates the concurrent dispatch pattern
async function* dispatchParallel(calls, context) {
  yield* boundedAll(
    calls.map(async function* (call) {
      context.markInProgress(call.id)
      yield* executeSingle(call, context)
      context.markComplete(call.id)
    }),
    MAX_CONCURRENCY,  // Default: 10
  )
}
```

The concurrency limit defaults to 10, configurable via `CLAUDE_CODE_MAX_TOOL_USE_CONCURRENCY`. Ten is generous -- you rarely see more than five or six tool calls in a single model response. The limit exists as a safety valve for pathological cases, not as a typical constraint.

### Serial Batches

Serial execution is straightforward. Each tool runs, its context modifiers are applied immediately, and the next tool sees the updated context:

```
for (const toolUse of toolUseMessages) {
  for await (const update of runToolUse(toolUse, /* ... */)) {
    if (update.contextModifier) {
      currentContext = update.contextModifier.modifyContext(currentContext)
    }
    yield { message: update.message, newContext: currentContext }
  }
}
```

This is the critical difference. Serial tools can change the world for subsequent tools. An Edit modifies a file; the next Read sees the modified version. A Bash command creates a directory; the next Bash command writes into it.

## The Streaming Tool Executor

Batch orchestration eliminates unnecessary serialization *after* the model's response arrives. But there is a bigger opportunity: the model's response takes time to stream. A typical multi-tool response might take 2-3 seconds to fully arrive. The first tool call is parseable after 500 milliseconds. Why wait for the remaining 2 seconds?

The `StreamingToolExecutor` class implements speculative execution. As the model streams its response, each `tool_use` block is handed to the executor the moment it is fully parsed. The executor starts running it immediately -- while the model is still generating the next tool call.

```
┌──────────────────────────────────────────────────────────────────┐
│           SEQUENTIAL vs STREAMING TOOL EXECUTION                 │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  SEQUENTIAL (3.1s total):                                        │
│  ├─ Model streams response (2.5s)                                │
│  ├─ Tool 1 (0.2s)                                                │
│  ├─ Tool 2 (0.3s)                                                │
│  └─ Tool 3 (0.1s)                                                │
│                                                                  │
│  STREAMING EXECUTOR (2.6s total):                                │
│  ├─ Model streams response (2.5s)                                │
│  │   ├─ Tool 1 starts at 0.5s, completes at 0.7s               │
│  │   └─ Tool 2 starts at 1.2s, completes at 1.5s               │
│  └─ Tool 3 drain after stream (0.1s)                             │
│                                                                  │
│  Savings: 16% wall-clock time                                    │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

## Tool Concurrency Properties

Each built-in tool declares its concurrency characteristics through the `isConcurrencySafe()` method:

| Tool | Concurrency Safe | Condition | Rationale |
|---|---|---|---|
| **Read** | Always | -- | Pure read. No side effects. |
| **Grep** | Always | -- | Pure read. Wraps ripgrep. |
| **Glob** | Always | -- | Pure read. File listing. |
| **Fetch** | Always | -- | HTTP GET. No local side effects. |
| **WebSearch** | Always | -- | API call to search provider. |
| **Bash** | Sometimes | Read-only commands only | `isReadOnly()` parses the command and classifies subcommands. |
| **Edit** | Never | -- | Modifies files. Two concurrent edits to the same file corrupt it. |
| **Write** | Never | -- | Creates or overwrites files. Same corruption risk. |
| **NotebookEdit** | Never | -- | Modifies `.ipynb` files. |

## Apply This

The concurrency patterns in Claude Code generalize to any system that orchestrates multiple independent operations. Three principles are worth extracting.

**Partition by safety, not by type.** The `isConcurrencySafe(input)` method receives the parsed input, not just the tool name. This per-invocation classification is more precise than a static "this tool type is always safe" declaration.

**Speculative execution during I/O waits.** The streaming executor starts tools while the API response is still arriving. The same pattern applies anywhere you have a slow producer and fast consumers.

**Preserve submission order in results.** Yielding results in completion order is tempting -- it minimizes latency to first result. But if the consumer expects results in a specific order, reordering them creates confusion.

# Chapter 8: Spawning Sub-Agents

## The Multiplication of Intelligence

A single agent is powerful. It can read files, edit code, run tests, search the web, and reason about the results. But there is a hard ceiling on what one agent can do in a single conversation: the context window fills up, the task branches in directions that demand different capabilities, and the serial nature of tool execution becomes a bottleneck. The solution is not a bigger model. It is more agents.

Claude Code's sub-agent system lets the model request help. When the parent agent encounters a task that would benefit from delegation -- a codebase search that should not pollute the main conversation, a verification pass that demands adversarial thinking, a set of independent edits that could run in parallel -- it calls the `Agent` tool. That call spawns a child: a fully independent agent with its own conversation loop, its own tool set, its own permission boundary, and its own abort controller. The child does its work and returns a result. The parent never sees the child's internal reasoning, only the final output.

## The runAgent Lifecycle

`runAgent()` in `runAgent.ts` is an async generator that drives a sub-agent's entire lifecycle. It yields `Message` objects as the agent works. Every sub-agent -- fork, built-in, custom, coordinator worker -- flows through this single function.

The function signature reveals the complexity of the problem:

```
export async function* runAgent({
  agentDefinition,       // What kind of agent
  promptMessages,        // What to tell it
  toolUseContext,        // Parent's execution context
  canUseTool,           // Permission callback
  isAsync,              // Background or blocking?
  canShowPermissionPrompts,
  forkContextMessages,  // Parent's history (fork only)
  querySource,          // Origin tracking
  override,             // System prompt, abort controller, agent ID overrides
  model,                // Model override from caller
  maxTurns,             // Turn limit
  availableTools,       // Pre-assembled tool pool
  allowedTools,         // Permission scoping
  onCacheSafeParams,    // Callback for background summarization
  useExactTools,        // Fork path: use parent's exact tools
  worktreePath,         // Isolation directory
  description,          // Human-readable task description
  // ...
}: { ... }): AsyncGenerator<Message, void>
```

Seventeen parameters. Each one represents a dimension of variation that the lifecycle must handle.

## The Fifteen Steps

Here are the fifteen steps of the `runAgent` lifecycle:

### Step 1: Model Resolution

The resolution chain is: **caller override** > **agent definition** > **parent model** > **default**. The caller (the parent model) can override the agent definition's preference by passing a `model` parameter in the tool call.

### Step 2: Agent ID Creation

Agent IDs follow the pattern `agent-<hex>` where the hex part is derived from `crypto.randomUUID()`. The override path exists for resumed agents that need to keep their original ID for transcript continuity.

### Step 3: Context Preparation

For fork agents, the parent's entire conversation history is cloned into `contextMessages`. A critical filter strips any `tool_use` blocks that lack matching `tool_result` blocks.

### Step 4: CLAUDE.md Stripping

Read-only agents like Explore and Plan have `omitClaudeMd: true` in their definitions. Dropping CLAUDE.md saves billions of tokens per week across the fleet.

### Step 5: Permission Isolation

Each agent gets a custom `getAppState()` wrapper that overlays its permission configuration onto the parent's state. Four concerns are layered: permission mode cascade, prompt avoidance, automated check ordering, and tool permission scoping.

### Step 6: Tool Resolution

Fork agents use `useExactTools: true`, which passes the parent's tool array through unchanged. This is a cache optimization -- byte-identical prefixes maximize prompt cache hits.

### Step 7: System Prompt

Fork agents receive the parent's pre-rendered system prompt via `override.systemPrompt`. Recomputing the system prompt could diverge due to GrowthBook feature flag transitions.

### Step 8: Abort Controller Isolation

Three behaviors: override (for resumed agents), async agents get a new independent controller, sync agents share the parent's controller.

### Step 9: Hook Registration

Agent definitions can declare their own hooks in frontmatter. These hooks are scoped to the agent's lifecycle via the `agentId`.

### Step 10: Skill Preloading

Agent definitions can specify `skills: ["my-skill"]` in their frontmatter. Loaded skills become user messages prepended to the agent's conversation.

### Step 11: MCP Initialization

Agents can define their own MCP servers in frontmatter, additive to the parent's clients. Only newly created (inline) clients are cleaned up when the agent finishes.

### Step 12: Context Creation

`createSubagentContext()` assembles the new `ToolUseContext`. Key isolation decisions: sync agents share `setAppState`, async agents get isolated `setAppState`, both share `setResponseLength`.

### Step 13: Cache-Safe Params Callback

This callback is consumed by background summarization. The params are "cache-safe" because they produce the same API request prefix the agent is using.

### Step 14: The Query Loop

The same `query()` function from Chapter 5 drives the sub-agent's conversation. Each yielded message is recorded to a sidechain transcript via `recordSidechainTranscript()`.

### Step 15: Cleanup

The `finally` block runs on normal completion, abort, or error. Every subsystem the agent touched gets cleaned up: MCP connections, hooks, cache tracking, file state, perfetto tracing, todo entries, and orphaned shell processes.

## Built-In Agent Types

| Agent | Model | Tools | Context | Sync/Async | Purpose |
|---|---|---|---|---|---|
| General-Purpose | Default | All | Full | Either | Workhorse delegation |
| Explore | Haiku | Read-only | Stripped | Sync | Fast, cheap search |
| Plan | Inherit | Read-only | Stripped | Sync | Architecture design |
| Verification | Inherit | Read-only | Full | Always async | Adversarial testing |
| Guide | Haiku | Read + Web | Dynamic | Sync | Documentation lookup |
| Statusline | Sonnet | Read + Edit | Minimal | Sync | Config task |

## Apply This: Designing Agent Types

The built-in agents demonstrate a pattern language for agent design. The design space breaks down into five dimensions.

**Dimension 1: What Can It See?** The combination of `omitClaudeMd`, git status stripping, and skill preloading controls the agent's awareness.

**Dimension 2: What Can It Do?** The `tools` and `disallowedTools` fields set hard boundaries.

**Dimension 3: How Does It Interact with the User?** The `permissionMode` and `canShowPermissionPrompts` settings determine whether the agent asks for permission, auto-denies, or bubbles prompts.

**Dimension 4: How Does It Relate to the Parent?** Sync agents block the parent and share its state. Async agents run independently. Fork agents inherit the full conversation context.

**Dimension 5: How Expensive Is It?** The model choice, thinking config, and context size all contribute to cost.

# Chapter 9: Fork Agents and the Prompt Cache

## The Ninety-Five Percent Insight

When a parent agent spawns five child agents in parallel, the overwhelming majority of each child's API request is identical. The system prompt is the same. The tool definitions are the same. The conversation history is the same. The assistant message that triggered the spawns is the same. The only thing that differs is the final directive: "you handle the database migration," "you write the tests," "you update the docs."

On a typical fork with a warm conversation, the shared prefix might be 80,000 tokens. The per-child directive might be 200 tokens. That is 99.75% overlap. Anthropic's prompt cache gives a 90% discount on cached input tokens. If you can make those 80,000 tokens hit the cache for children 2 through 5, you just cut the input cost of those four requests by 90%.

The catch is that prompt caching is byte-exact. Not "similar enough." Not "semantically equivalent." The bytes must match, character for character, from the first byte of the system prompt through to the last byte before the per-child content diverges.

## What a Fork Child Inherits

A fork agent inherits four things from its parent, and it inherits them by reference or byte-exact copy, not by recomputation.

**1. The system prompt.** Not regenerated -- threaded. The parent's already-rendered system prompt bytes are passed via `override.systemPrompt`.

**2. The tool definitions.** The fork agent definition declares `tools: ['*']`, but with the `useExactTools` flag set to true, the child receives the parent's assembled tool array directly.

**3. The conversation history.** Every message the parent has exchanged with the API is cloned into the child's context via `forkContextMessages`.

**4. The thinking configuration and model.** The fork definition specifies `model: 'inherit'`, which resolves to the parent's exact model.

## The Byte-Identical Prefix Trick

Fork agents achieve byte-identical prefixes by ensuring three layers are frozen:

**Layer 1: System prompt via threading, not recomputation.** Threading the rendered bytes eliminates the entire class of divergence from GrowthBook feature flags transitioning between cold and warm state.

**Layer 2: Tool definitions via exact passthrough.** The `useExactTools` flag passes the parent's tool pool unchanged. Same tools, same order, same serialization.

**Layer 3: Message array construction.** The `buildForkedMessages()` function constructs the final two messages with placeholder tool results that are identical across all children.

## Recursive Fork Prevention

The fork child keeps the Agent tool in its tool pool. It has to -- removing it would change the serialized tool array and bust the prompt cache. But if the child actually invokes the Agent tool without `subagent_type`, the fork path would trigger again.

Two guards prevent this:

**Primary guard: querySource check.** When a fork child is spawned, its `context.options.querySource` is set to `'agent:builtin:fork'`. The `call()` method checks this before allowing the fork path.

**Fallback guard: message scanning.** A fallback scans message history for the boilerplate XML tag. This catches edge cases where `querySource` was not properly threaded.

## The Economics

Consider a concrete scenario. A developer asks Claude Code to refactor a module. The parent agent analyzes the codebase, forms a plan, and dispatches five fork children in parallel.

At this point in the conversation, the shared context is substantial:

- System prompt: ~4,000 tokens
- Tool definitions (40+ tools): ~12,000 tokens
- Conversation history (analysis + planning): ~30,000 tokens
- Assistant message with five tool_use blocks: ~2,000 tokens

- Placeholder tool results: ~500 tokens

Total shared prefix: ~48,500 tokens. Per-child directive: ~200 tokens.

With fork (byte-identical prefixes): Children 2-5 get 48,500 tokens at 10% price (cache hit) + 200 tokens at full price each. Effective cost for children 2-5: ~5,050 tokens equivalent each.

## Apply This: Designing for Prompt Cache Efficiency

**1. Thread rendered prompts, do not recompute.** Capture the rendered result and pass it to children by value.

**2. Freeze the tool array.** Consider keeping the full tool set and using runtime guards instead of compile-time removal.

**3. Maximize the shared prefix, minimize the per-child suffix.** Structure your message array so that everything shared comes first.

**4. Use constant placeholders for variable content.** Use identical placeholder strings across all children rather than actual (divergent) results.

**5. Measure the break-even.** Cache sharing has overhead: larger context windows per child, runtime guards instead of static safety.

# Chapter 10: Tasks, Coordination, and Swarms

## The Limits of a Single Thread

Chapter 8 showed how to create a sub-agent -- the fifteen-step lifecycle that builds an isolated execution context from an agent definition. Chapter 9 showed how to make parallel spawns economical through prompt cache exploitation. But creating agents and managing agents are different problems. This chapter addresses the second.

A single agent loop -- one model, one conversation, one tool at a time -- can accomplish a remarkable amount of work. But it hits a ceiling. The ceiling is not intelligence. It is parallelism and scope.

## The Task State Machine

Every background operation in Claude Code -- a shell command, a sub-agent, a remote session, a workflow script -- is tracked as a *task*. The task abstraction lives in `Task.ts` and provides the unified state model that the rest of the orchestration layer builds on.

### Seven Types

The system defines seven task types, each representing a different execution model:

- `local_bash` -- background shell commands
- `local_agent` -- background sub-agents
- `remote_agent` -- remote sessions
- `in_process_teammate` -- swarm teammates
- `local_workflow` -- workflow script executions
- `monitor_mcp` -- MCP server monitors
- `dream` -- speculative background thinking

Each type gets a single-character ID prefix for instant visual identification:

| Type | Prefix | Example ID |
|---|---|---|
| `local_bash` | `b` | `b4k2m8x1` |
| `local_agent` | `a` | `a7j3n9p2` |
| `remote_agent` | `r` | `r1h5q6w4` |
| `in_process_teammate` | `t` | `t3f8s2v5` |
| `local_workflow` | `w` | `w6c9d4y7` |
| `monitor_mcp` | `m` | `m2g7k1z8` |
| `dream` | `d` | `d5b4n3r6` |

## Five Statuses

```
┌──────────────────────────────────────────────────────────────────┐
│                        TASK STATUS FLOW                          │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│                                                                  │
│      ┌─────────┐                                                 │
│      │ pending │──────────────────────┐                          │
│      └─────────┘                      │                          │
│           │ execution starts          │                          │
│           ▼                           │                          │
│      ┌─────────┐                      │                          │
│      │ running │──────────────────────┼──────────────────────┐   │
│      └─────────┘                      │   │                  │   │
│           │                           │   │                  │   │
│           │ normal finish             │   │ error            │   │ abort
│           ▼                           │   ▼                  │   │
│      ┌─────────┐                      │ ┌──────┐             │   │
│      │completed│                      │ │ failed│            │   │
│      └─────────┘                      │ └──────┘             │   │
│                                       │                      │   │
│                                       └──────────────────────┼───┘
│                                                              │
│                                                              ▼
│                                                          ┌──────┐
│                                                          │ killed│
│                                                          └──────┘
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

# Coordinator Mode

Coordinator mode transforms Claude Code from a single agent with background helpers into a true manager-worker architecture. It is the most opinionated orchestration pattern in the system.

## The 370-Line System Prompt

The coordinator system prompt is, line for line, the most instructive document in the codebase about how to use LLMs for orchestration. The key teachings:

**"Never delegate understanding."** This is the central thesis. The coordinator must synthesize research findings into specific prompts with file paths, line numbers, and exact changes.

**"Parallelism is your superpower."** The prompt establishes a clear concurrency model. Read-only tasks run freely in parallel. Write-heavy tasks serialize per file set.

### Task Workflow Phases

```
┌──────────────────────────────────────────────────────────────────┐
│                    COORDINATOR WORKFLOW PHASES                   │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   ┌──────────────┐        research results      ┌──────────────┐  │
│   │ Research     │──────────────────────────────▶│ Synthesis    │  │
│   └──────────────┘                              └──────────────┘  │
│          │                                            │          │
│          │ 3-5 workers in parallel                    │ Coordinator only│
│          │ read files, run tests                      │ no workers  │
│          │                                            │          │
│          ▼                                            ▼          │
│   ┌──────────────┐        specific instructions  ┌──────────────┐  │
│   │              │──────────────────────────────▶│Implementation│  │
│   └──────────────┘                              └──────────────┘  │
│          │                                            │          │
│          │ Workers per file set                       │          │
│          │ disjoint changes                           │          │
│          │                                            │          │
│          │ test results                               │          │
│          ▼                                            ▼          │
│   ┌──────────────┐                              ┌──────────────┐  │
│   │              │◀──────────────────────────────│Verification│  │
│   └──────────────┘                              └──────────────┘  │
│          │                                            │          │
│          │ Workers run tests                          │          │
│          │ verify changes                             │          │
│          │                                            │          │
│          └────────────────────────────────────────────┘          │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

## The Swarm System

Coordinator mode is hierarchical: one manager, many workers, top-down control. The swarm system is the peer-to-peer alternative -- multiple Claude Code instances working as a team, with a leader coordinating multiple teammates through message passing.

### The Mailbox

Teammates communicate via a file-based mailbox system. When `SendMessage` targets a teammate, the message is written to the recipient's mailbox file on disk:

```
await writeToMailbox(recipientName, {
  from: senderName,
  text: content,
  summary,
  timestamp: new Date().toISOString(),
  color: senderColor,
}, teamName)
```

The file-based approach is deliberately simple. Files are durable (surviving process crashes), inspectable (you can `cat` a mailbox), and cheap (no infrastructure dependencies).

## SendMessage Routing

```
┌──────────────────────────────────────────────────────────────────┐
│                    SENDMESSAGE ROUTING                           │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  SendMessage(to: X)                                              │
│         │                                                        │
│         ▼                                                        │
│  ┌──────────────┐                                                │
│  │ starts with  │──YES──► Bridge relay                          │
│  │  'bridge:'   │         (cross-machine via Remote Control)    │
│  └──────────────┘                                                │
│         │ NO                                                     │
│         ▼                                                        │
│  ┌──────────────┐                                                │
│  │ starts with  │──YES──► Unix Domain Socket                    │
│  │   'uds:'     │         (local inter-process)                 │
│  └──────────────┘                                                │
│         │ NO                                                     │
│         ▼                                                        │
│  ┌──────────────────────────────────────────┐                   │
│  │ found in agentNameRegistry or AppState?  │                   │
│  └──────────────────────────────────────────┘                   │
│                 │             │                                  │
│                 │             │                                  │
│            YES, running  YES, terminal                           │
│                 │             │                                  │
│                 ▼             ▼                                  │
│         Queue pending   Auto-resume agent                        │
│         message         from disk transcript                     │
│         (delivered at                                           │
│          tool-round                                             │
│          boundary)                                               │
│                 │                                                │
│                 │ NO                                             │
│                 ▼                                                │
│          ┌──────────────┐                                        │
│          │ team context │──YES──► Write to mailbox file         │
│          │   active?    │                                        │
│          └──────────────┘                                        │
│                 │ NO                                              │
│                 ▼                                                 │
│         Error: recipient not found                                │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

## Apply This

**Simple delegation** (Agent tool with `run_in_background: true`) is appropriate when the parent has one or two independent tasks to offload.

**Coordinator mode** is appropriate when the problem decomposes into a research phase, a synthesis phase, and an implementation phase.

**Swarm teams** are appropriate for long-running collaborative sessions where agents need peer-to-peer communication.

# Chapter 11: Memory -- Learning Across Conversations

## The Stateless Problem

Every chapter so far has described machinery that exists within a single session. The agent loop runs, tools execute, sub-agents coordinate, and when the process exits, all of it vanishes. The next conversation starts with the same system prompt, the same tool definitions, the same model -- and zero knowledge of what happened before.

This is the fundamental limitation of a stateless architecture. A developer corrects the model's testing approach on Monday, and on Tuesday the model makes the same mistake. Without memory, every session starts at zero.

## The Four-Type Taxonomy

Not everything is worth remembering. The memory system constrains all memories to exactly four types:

- **User memories** record information about the person: their role, goals, responsibilities, expertise level.
- **Feedback memories** capture guidance about how to approach work -- both corrections and confirmations.
- **Project memories** record ongoing work context -- who is doing what, why, by when.
- **Reference memories** are bookmarks -- pointers to where information lives in external systems.

## The Recall Path

```
┌──────────────────────────────────────────────────────────────────┐
│                        MEMORY RECALL PATH                        │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  User submits query                                              │
│         │                                                        │
│         ▼                                                        │
│  startRelevantMemoryPrefetch                                     │
│  (fires async, parallel with main model)                         │
│         │                                                        │
│         ▼                                                        │
│  scanMemoryFiles                                                 │
│  reads all .md files, parses frontmatter, 30 lines max per file │
│         │                                                        │
│         ▼                                                        │
│  Filter already-surfaced paths                                   │
│         │                                                        │
│         ▼                                                        │
│  formatMemoryManifest                                            │
│  one line per file: type, name, date, description               │
│         │                                                        │
│         ▼                                                        │
│  Sonnet side-query receives:                                     │
│  - manifest                                                      │
│  - user query                                                    │
│  - recently-used tools                                           │
│         │                                                        │
│         ▼                                                        │
│  Sonnet returns up to 5 filenames                                │
│  (via structured JSON output)                                    │
│         │                                                        │
│         ▼                                                        │
│  Validate filenames against known set                            │
│  (catching hallucinated names)                                   │
│         │                                                        │
│         ▼                                                        │
│  Read selected files in full                                     │
│  attach as relevant_memories with staleness warnings            │
│         │                                                        │
│         ▼                                                        │
│  Collapse groups in UI                                           │
│  absorb attachments for rendering                                │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

## Staleness

The staleness system addresses a failure mode that emerged from real usage. Users reported that old memories -- containing file:line citations to code that had since changed -- were being asserted as fact by the model.

The solution is not expiration. Old memories are not deleted -- they may contain institutional knowledge valid for years. Instead, the system attaches age warnings. Memories from today or yesterday get no warning. Everything older gets a staleness caveat injected alongside the memory content.

## KAIROS Mode: Append-Only Daily Logs

Standard memory assumes discrete sessions. KAIROS mode (Claude Code's assistant mode) breaks this assumption -- sessions are long-lived, potentially running for days. The solution is architectural separation between capture and consolidation:

```
┌──────────────────────────────────────────────────────────────────┐
│                STANDARD vs KAIROS MEMORY MODES                   │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  STANDARD WRITE PATH:                                            │
│  ┌──────────────┐    ┌──────────────┐    ┌──────────────────┐   │
│  │Model observes│──►│Create memory │──►│Update MEMORY.md│   │
│  └──────────────┘    │   file       │    │     index        │   │
│                      └──────────────┘    └──────────────────┘   │
│                                                                  │
│  KAIROS MODE:                                                    │
│  ┌──────────────┐    ┌──────────────────────────┐               │
│  │Model observes│──►│Append timestamped bullet│               │
│  └──────────────┘    │    to daily log file     │               │
│                      └──────────────────────────┘               │
│                                  │                               │
│                                  ▼                               │
│                      ┌──────────────────────────┐               │
│                      │  /dream consolidation    │               │
│                      │  (background process)    │               │
│                      └──────────────────────────┘               │
│                                  │                               │
│                                  ▼                               │
│                      ┌──────────────────────────┐               │
│                      │  Read recent logs        │               │
│                      └──────────────────────────┘               │
│                                  │                               │
│                                  ▼                               │
│                      ┌──────────────────────────┐               │
│                      │  Merge into structured   │               │
│                      │      memories            │               │
│                      └──────────────────────────┘               │
│                                  │                               │
│                                  ▼                               │
│                      ┌──────────────────────────┐               │
│                      │  Update MEMORY.md index  │               │
│                      └──────────────────────────┘               │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

## Apply This: Designing Agent Memory

**Files beat databases for agent memory.** Files are inspectable, editable, and version-controllable. Transparency builds trust.

**Constrain what gets saved, not just how.** The derivability test -- can this knowledge be re-derived from the current project state? -- eliminates the majority of potential memories.

**Use an LLM for recall, not keywords or embeddings.** An LLM side-query understands context, reasons about what is already available in conversation, handles negation.

**Warn about staleness, do not expire.** Institutional knowledge may remain valid for years. Attaching age warnings lets the model treat old memories as hypotheses rather than facts.

# Chapter 12: Extensibility -- Skills and Hooks

## Two Dimensions of Extension

Every extensibility system answers two questions: what can the system do, and when does it do it. Claude Code separates them cleanly. Skills extend what the model can do. Hooks extend when and how things happen.

# Skills: Teaching the Model New Tricks

## Two-Phase Loading

```
┌──────────────────────────────────────────────────────────────────────┐
│                      SKILL LOADING PHASES                            │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  PHASE 1: STARTUP                                                    │
│  ┌──────────────────┐    ┌──────────────────┐    ┌──────────────┐   │
│  │Read SKILL.md     │──►│Extract YAML      │──►│Build system  │   │
│  │files from        │    │frontmatter       │    │prompt menu   │   │
│  │7 sources         │    │(name, desc,      │    │(model knows  │   │
│  │                  │    │ whenToUse)       │    │ skills exist)│   │
│  └──────────────────┘    └──────────────────┘    └──────────────┘   │
│                                │                                     │
│                                │ on invocation                       │
│                                ▼                                     │
│  PHASE 2: INVOCATION                                                 │
│  ┌──────────────────┐    ┌──────────────────┐    ┌──────────────┐   │
│  │User or model     │──►│getPromptForCommand│──►│Variable      │   │
│  │invokes           │    │executes          │    │substitution  │   │
│  │/skill-name       │    │                  │    │(ARGUMENTS,   │   │
│  │                  │    │                  │    │SKILL_DIR,    │   │
│  └──────────────────┘    └──────────────────┘    │SESSION_ID)   │   │
│                                │                  └──────────────┘   │
│                                │                       │            │
│                                ▼                       ▼            │
│                        ┌──────────────────┐    ┌──────────────┐    │
│                        │Inline shell      │──►│Content blocks│    │
│                        │execution         │    │injected into │    │
│                        │(unless MCP)      │    │conversation  │    │
│                        └──────────────────┘    └──────────────┘    │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

### Seven Sources with Priority

| Priority | Source | Location | Notes |
|---|---|---|---|
| 1 | Managed (Policy) | `<MANAGED_PATH>/.claude/skills/` | Enterprise-controlled |
| 2 | User | `~/.claude/skills/` | Personal, available everywhere |
| 3 | Project | `.claude/skills/` | Checked into version control |
| 4 | Additional Dirs | `<add-dir>/.claude/skills/` | Via `--add-dir` flag |
| 5 | Legacy Commands | `.claude/commands/` | Backwards-compatible |
| 6 | Bundled | Compiled into the binary | Feature-gated |
| 7 | MCP | MCP server prompts | Remote, untrusted |

## Hooks: Controlling When Things Happen

### The Five Most Important Lifecycle Events

| Event | Description |
|---|---|
| **PreToolUse** | Fires before every tool execution. Can block, modify input, auto-approve, or inject context. |
| **PostToolUse** | Fires after successful execution. Can inject context or replace MCP tool output entirely. |
| **Stop** | Fires before Claude concludes its response. A blocking hook forces continuation. |
| **SessionStart** | Fires at session beginning. Can set environment variables, override the first user message. |
| **UserPromptSubmit** | Fires when the user submits a prompt. Can block processing. |

## Exit Code Semantics

| Exit Code | Meaning | Blocks |
|---|---|---|
| 0 | Success, stdout parsed if JSON | No |
| 2 | Blocking error, stderr shown as system message | Yes |
| Other | Non-blocking warning, shown to user only | No |

## The Snapshot Security Model

```
┌──────────────────────────────────────────────────────────────────┐
│                    SNAPSHOT SECURITY MODEL                       │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  User ──▶ Open project                                           │
│             │                                                    │
│             ▼                                                    │
│        Read all hook configs                                     │
│             │                                                    │
│             ▼                                                    │
│        captureHooksConfigSnapshot()                              │
│             │                                                    │
│             ▼                                                    │
│         ┌──────────────────────────┐                             │
│         │   Hooks frozen in memory │                              │
│         │   (snapshot captured)    │                             │
│         └──────────────────────────┘                             │
│             │                                                    │
│             ▼                                                    │
│        Accept workspace trust                                    │
│             │                                                    │
│             ▼                                                    │
│        Normal operation begins                                   │
│             │                                                    │
│             │  (Attacker modifies .claude/settings.json)         │
│             │                                                    │
│             ▼                                                    │
│        executeHooks()                                            │
│             │                                                    │
│             ▼                                                    │
│         ┌──────────────────────────┐                             │
│         │ Reads from frozen        │                             │
│         │ snapshot, NOT from       │                             │
│         │ filesystem!              │                             │
│         └──────────────────────────┘                             │
│             │                                                    │
│             ▼                                                    │
│        Malicious changes IGNORED                                 │
│                                                                  │
│  captureHooksConfigSnapshot() is called ONCE during startup.     │
│  From that point, executeHooks() reads from the snapshot,        │
│  never re-reading settings files implicitly.                     │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

# Chapter 13: The Terminal UI

## Rendering in the Terminal

Claude Code runs in a terminal. Not a browser, not a native GUI framework, not a web view. A terminal. This constraint shapes every design decision in the UI layer. The terminal is a grid of characters, each with foreground and background colors, bold/italic/underline attributes, and nothing else. There are no CSS animations, no SVG paths, no GPU-accelerated transforms. Just characters, rendered one at a time.

## The Custom React Renderer

Claude Code uses Ink, a custom React renderer for terminals. But Ink is not used off-the-shelf. The codebase maintains a fork with optimizations for Claude Code's specific workload:

- **Packed typed arrays** for character cells, reducing memory overhead from objects to raw bytes.
- **Pool-based string interning** to deduplicate repeated strings (like ANSI escape sequences).
- **Cell-level diffing** to minimize terminal writes -- only changed cells are re-rendered.
- **Lazy text measurement** to avoid expensive width calculations for off-screen content.

The optimizations are not premature. They are driven by profiling. A typical Claude Code session generates thousands of output lines per minute. Without the optimizations, the UI would stutter.

## Component Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                      COMPONENT ARCHITECTURE                      │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│                              ┌─────────┐                         │
│                              │   App   │                         │
│                              └────┬────┘                         │
│                   ┌───────────────┼───────────────┐              │
│                   │               │               │              │
│                   ▼               ▼               ▼              │
│              ┌─────────┐    ┌────────────┐    ┌─────────┐        │
│              │ Header  │    │Main Content│    │ Footer  │        │
│              └─────────┘    └────────────┘    └─────────┘        │
│                                   │                              │
│                          ┌────────┴────────┐                     │
│                          │                 │                     │
│                          ▼                 ▼                     │
│                   ┌───────────┐      ┌──────────┐                │
│                   │MessageList│      │InputArea │                │
│                   └───────────┘      └──────────┘                │
│                          │                                       │
│              ┌───────────┼───────────┐                           │
│              │           │           │                           │
│              ▼           ▼           ▼                           │
│         ┌────────┐  ┌─────────┐  ┌──────────┐                   │
│         │UserMsg │  │AsstMsg  │  │ToolResult│                   │
│         └────────┘  └─────────┘  └──────────┘                   │
│                          │                                       │
│              ┌───────────┼───────────┐                           │
│              │           │           │                           │
│              ▼           ▼           ▼                           │
│         ┌────────┐  ┌─────────┐  ┌──────────┐                   │
│         │TextBlk │  │CodeBlk  │  │ToolUseBlk│                   │
│         └────────┘  └─────────┘  └──────────┘                   │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

## Message Rendering

Each message type has its own rendering logic:

**User messages** are simple: the user's text, possibly with attachments. They render in a distinct color to distinguish them from the assistant.

**Assistant messages** are complex. They can contain text, thinking blocks, and tool_use blocks. Each block type has its own rendering: text flows naturally, thinking blocks are collapsible and styled differently, tool_use blocks show the tool name and input in a compact format.

**Tool results** show the output of tool execution. Bash output uses syntax highlighting. File edits show a diff view. Search results show matches with context lines.

## Streaming Output

The most challenging part of the UI is streaming output. As the model generates tokens, they appear on screen in real time. This requires:

1. **Incremental parsing** of the model's response to extract content blocks as they complete.
2. **Efficient re-rendering** that updates only the changed parts of the output.
3. **Smooth scrolling** that follows the output without jumping or flickering.

The streaming implementation uses a virtual list for long conversations. Only visible messages are rendered; off-screen messages are represented by placeholder height calculations. This keeps the UI responsive even with thousands of messages.

## Syntax Highlighting

Code blocks are syntax-highlighted using a custom highlighter optimized for terminal output. The highlighter:

- Uses a compact color palette suitable for terminals (16 colors, not millions).
- Highlights incrementally as code streams in, not waiting for the full block.
- Falls back to plain text for unknown languages, never failing to render.

The highlighting is done via React Suspense, loading the highlighter only when the first code block appears. This keeps initial bundle size small.

## Responsive Design in a Terminal

Terminals resize. The UI must adapt:

- **Line wrapping** recalculates when the terminal width changes.

- **Column layouts** collapse to single column on narrow terminals.
- **Truncation** shows "... and 47 more" for long lists instead of overflowing.

The resize handler is debounced to avoid excessive re-renders during rapid resizing.

## Apply This: Designing Terminal UIs

**Profile before optimizing.** The Ink optimizations were driven by real measurements of frame time and memory usage. Premature optimization without data is guesswork.

**Use virtual lists for long content.** Rendering thousands of DOM nodes (or terminal cells) is slow. Virtual lists keep the active set small.

**Stream, do not buffer.** Users perceive latency in hundreds of milliseconds. Buffering a complete response before displaying it feels slow. Incremental rendering feels fast.

**Handle resize gracefully.** Terminal users resize windows. The UI should adapt without breaking layout or losing content.

# Chapter 14: Input and Interaction

## The Input Loop

While the agent loop handles the model's output, a separate input loop handles the user's input. The two loops run concurrently, with the input loop able to interrupt the agent loop at any time.

The input loop follows this pattern:

```
while (true) {
  // 1. Wait for user input
  const input = await readInput();

  // 2. Parse the input
  const parsed = parseInput(input);

  // 3. Handle special commands
  if (isCommand(parsed)) {
    await executeCommand(parsed);
    continue;
  }

  // 4. Submit to the agent loop
  await submitToAgent(parsed);
}
```

## Input Modes

Claude Code supports multiple input modes:

**Direct input** is the default. The user types, presses Enter, and the input is submitted to the agent.

**Multiline input** is triggered by Shift+Enter or by starting with a special marker. This allows pasting code blocks or long prompts without sending prematurely.

**Command mode** is triggered by the `/` prefix. Commands like `/clear`, `/compact`, `/help` are handled by the input loop, not sent to the agent.

**File attachment** is triggered by `@` followed by a path. The file is read and attached to the message as context.

## Interrupt Handling

The user can interrupt the agent at any time:

- **Ctrl+C** aborts the current operation. If the model is streaming, it stops. If a tool is running, it is cancelled.
- **Ctrl+D** exits the session gracefully, saving state.
- **Escape** cancels the current input without submitting it.

Interrupt handling uses an AbortController that is shared between the input loop and the agent loop. When Ctrl+C is pressed, the controller is aborted, and all async operations check the signal and terminate cleanly.

## History and Recall

The input loop maintains a history of previous inputs:

- **Up/Down arrows** navigate through history.
- **Ctrl+R** triggers reverse search through history.
- **History is persisted** to disk and restored on session resume.

The history is deduplicated -- consecutive identical inputs are collapsed to one entry. This prevents the history from filling up with repeated commands.

## Autocomplete

The input loop provides autocomplete for:

- **File paths** after `@` or in tool arguments.
- **Commands** after `/`.
- **Skill names** after `/skill`.

Autocomplete is triggered by Tab and shows a dropdown of suggestions. The suggestions are filtered as the user types, and Enter selects the highlighted option.

## Apply This: Designing Input Systems

**Separate input from processing.** The input loop should be lightweight and responsive, even when the processing loop is busy. This requires true concurrency, not just async/await.

**Support interruption.** Users need a way to stop a long-running operation. Ctrl+C is the universal signal; respect it.

**Persist what matters.** Input history is worth saving across sessions. Partial input is not -- if the user didn't submit it, they probably didn't want it.

**Make common operations fast.** Autocomplete for file paths and commands saves keystrokes. History navigation saves retyping. These small conveniences add up.

# Chapter 15: MCP -- The Universal Tool Protocol

## The Integration Problem

Every agentic system faces the same problem: how do you add new capabilities? The naive answer is to write more code -- more tools, more integrations, more maintenance. But that does not scale. Every new integration requires code changes, testing, deployment. The system becomes a monolith, tightly coupled to every tool it supports.

MCP (Model Context Protocol) is the answer. It is a protocol, not a library. Any tool that speaks MCP can be used by any agent that speaks MCP. The integration is decoupled: the tool author maintains the tool, the agent author maintains the agent, and the protocol ensures they can work together.

## The Three Capabilities

MCP defines three capabilities that a server can expose:

**Tools** are actions the model can invoke. Each tool has a name, description, and input schema. When the model calls a tool, the agent forwards the call to the MCP server and returns the result.

**Resources** are data the model can read. Each resource has a URI and content type. When the model needs context, it can read resources from MCP servers.

**Prompts** are templates the model can use. Each prompt has a name and content. Prompts from MCP servers appear as skills in Claude Code.

## The Connection Lifecycle

```
┌──────────────────────────────────────────────────────────────────┐
│                    MCP CONNECTION LIFECYCLE                      │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  Agent                            MCP Server                     │
│    │                                  │                          │
│    │─── Initialize ─────────────────►│                          │
│    │    (protocol version,            │                          │
│    │     capabilities)                │                          │
│    │                                  │                          │
│    │◄── Initialize response ────────│                          │
│    │    (server capabilities)         │                          │
│    │                                  │                          │
│    │─── Initialized notification ─►│                          │
│    │                                  │                          │
│    │         Connection established   │                          │
│    │                                  │                          │
│    │─── tools/list ─────────────────►│                          │
│    │◄── List of available tools ───│                          │
│    │                                  │                          │
│    │─── resources/list ─────────────►│                          │
│    │◄── List of available resources│                          │
│    │                                  │                          │
│    │─── prompts/list ───────────────►│                          │
│    │◄── List of available prompts ─│                          │
│    │                                  │                          │
│    │         Discovery complete       │                          │
│    │                                  │                          │
│    │─── tools/call ─────────────────►│                          │
│    │    (tool_name, arguments)        │                          │
│    │◄── Tool result ────────────────│                          │
│    │                                  │                          │
│    │─── resources/read ─────────────►│                          │
│    │    (uri)                         │                          │
│    │◄── Resource content ──────────│                          │
│    │                                  │                          │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

## Security Model

MCP servers run as separate processes. This provides isolation: a buggy or malicious MCP server cannot crash the main agent. But it also creates a security boundary that must be managed.

Claude Code implements several security measures:

- **Explicit installation** -- MCP servers must be explicitly added via `claude mcp add`.

- **Permission checks** -- MCP tool calls go through the same permission system as built-in tools.
- **Output sanitization** -- MCP tool results are wrapped and can be modified by PostToolUse hooks.
- **Timeout enforcement** -- MCP calls have a timeout to prevent hung servers from blocking the agent.

## Error Handling

MCP servers can fail in several ways:

- **Connection failure** -- the server process crashes or is not reachable.
- **Protocol error** -- the server sends malformed JSON or violates the protocol.
- **Tool error** -- the tool execution fails (e.g., file not found).
- **Timeout** -- the server takes too long to respond.

Each failure mode has a specific error type that the agent handles appropriately. Connection failures trigger reconnection attempts. Protocol errors log diagnostics. Tool errors are reported to the model as tool results. Timeouts abort the call and report a timeout error.

## Apply This: Designing Protocol Integrations

**Decouple via protocols, not libraries.** A protocol lets tool authors and agent authors evolve independently. A library creates a dependency that must be updated in lockstep.

**Isolate untrusted code.** MCP servers run in separate processes. Even if you trust the author, bugs happen. Process isolation contains the blast radius.

**Make security explicit.** Users should know what capabilities they are granting to external tools. Explicit installation and permission checks create that visibility.

**Handle all failure modes.** External systems fail in ways you cannot predict. Connection failures, protocol errors, timeouts -- each needs a specific handling strategy.

# Chapter 16: Remote Control and Cloud Execution

## The Local-Only Limitation

Claude Code is designed to run locally. It reads local files, runs local shell commands, edits local code. This is the right default for most use cases -- local execution is fast, private, and does not require network access.

But there are scenarios where local-only is a limitation:

- **Remote development** -- the code lives on a server, not your laptop.
- **Cloud resources** -- you need access to GPUs, large datasets, or specialized hardware.
- **Collaboration** -- multiple developers want to share a Claude Code session.
- **CI/CD integration** -- you want Claude Code to run in automated pipelines.

## The Remote Control Architecture

Remote Control extends Claude Code's reach beyond the local machine. It is not a separate product -- it is the same Claude Code, running remotely, controlled from your local terminal.

```
┌──────────────────────────────────────────────────────────────────────┐
│                      REMOTE CONTROL ARCHITECTURE                     │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   ┌──────────────────────┐          ┌──────────────────────────────┐│
│   │   LOCAL MACHINE      │          │       REMOTE MACHINE         │
│   │                      │          │                              │
│   │  ┌────────────────┐  │          │  ┌────────────────────────┐  │
│   │  │  Terminal UI   │  │          │  │  Claude Code Server    │  │
│   │  │                │  │          │  │                        │  │
│   │  │ • Input        │◄─┼─WebSocket─┼─►│ • Agent Loop         │  │
│   │  │ • Display      │  │   (TLS)  │  │ • State Manager      │  │
│   │  │ • Rendering    │  │          │  │ • Renderer           │  │
│   │  └────────────────┘  │          │  └────────────────────────┘  │
│   │          ▲           │          │              │               │
│   │          │           │          │              ▼               │
│   │  ┌────────────────┐  │          │  ┌────────────────────────┐  │
│   │  │ Input Handler  │  │          │  │   External Access      │  │
│   │  │                │──┼──────────┼─►│                        │  │
│   │  │ • Keystrokes   │  │          │  │ • Filesystem Access    │  │
│   │  │ • Paste        │  │          │  │ • Bash Execution       │  │
│   │  │ • Resize       │  │          │  │ • API Calls            │  │
│   │  └────────────────┘  │          │  └────────────────────────┘  │
│   │                      │          │                              │
│   └──────────────────────┘          └──────────────────────────────┘│
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

## Connection Establishment

The remote connection is established via a WebSocket tunnel:

1. The user runs `claude remote connect <host>`.
2. The local CLI establishes a WebSocket connection to the remote host.
3. The remote host starts a Claude Code server process if one is not already running.
4. The server authenticates the connection and begins proxying terminal I/O.

The WebSocket connection is encrypted (wss://) and authenticated. The authentication uses the same credentials as the Anthropic API, ensuring that only authorized users can connect.

## Protocol

Once connected, the protocol is simple: terminal input from the local machine is sent to the remote server, and terminal output from the remote server is sent back. The protocol handles:

- **Input forwarding** -- keystrokes, paste events, resize events.
- **Output streaming** -- the remote server's terminal output.
- **File transfer** -- uploading and downloading files.
- **Heartbeat** -- keeping the connection alive during idle periods.

The protocol is stateless with respect to the conversation -- the remote server maintains the full state, and the local terminal is just a view into that state.

## Security Considerations

Remote execution introduces security risks that local execution does not:

- **Network exposure** -- the remote server must accept connections from the internet.
- **Credential exposure** -- the remote server needs access to the Anthropic API key.
- **Data exposure** -- code and conversation history traverse the network.

Claude Code mitigates these risks:

- **End-to-end encryption** -- all traffic is encrypted via TLS.
- **Short-lived tokens** -- authentication tokens expire quickly.
- **Audit logging** -- all remote connections are logged.
- **Optional** -- remote control is disabled by default and must be explicitly enabled.

## Apply This: Designing Remote Systems

**Start local, extend remote.** The local experience is the foundation. Remote is an extension, not a replacement. The same code runs in both modes.

**Use standard protocols.** WebSocket is widely supported and well-understood. Do not invent a new wire protocol unless necessary.

**Security by default.** Remote features should be opt-in, not opt-out. Users should understand the risks before enabling remote access.

**Minimize state in the client.** The remote server is the source of truth. The local client is a thin view. This simplifies recovery from disconnections and enables seamless handoff between devices.

# Chapter 17: Performance -- Every Millisecond and Token Counts

## The Performance Mindset

Performance is not an afterthought in Claude Code. It is a first-class concern, baked into the architecture from the start. Every subsystem has been optimized, measured, and re-optimized. The result is a system that feels responsive even under heavy load.

This chapter is a tour of the performance optimizations across the codebase. It is not exhaustive -- new optimizations are added regularly -- but it covers the major techniques and their rationale.

## Startup Optimization

Chapter 2 described the bootstrap process. The key insight is that CPU-bound work (module loading) and I/O-bound work (keychain reads, network handshakes) can happen in parallel.

Specific techniques:

- **Fire-and-forget I/O** -- keychain reads and API preconnection start at module scope, before imports complete.
- **Deferred loading** -- heavy subsystems (OpenTelemetry, syntax highlighting, MCP servers) load on first use, not at startup.
- **Fast-path dispatch** -- simple commands (`--version`, `mcp`) exit before the full REPL initializes.

The result: interactive REPL in under 2 seconds from cold start.

## Rendering Optimization

Chapter 13 described the custom React renderer. The optimizations are:

- **Packed typed arrays** -- character cells stored as raw bytes, not objects.
- **String interning** -- repeated strings (ANSI codes) deduplicated via a pool.
- **Cell-level diffing** -- only changed cells are re-rendered.

- **Virtual lists** -- only visible messages are rendered.

The result: 60fps streaming output even with thousands of messages.

## API Optimization

Chapter 4 described the API layer. The optimizations are:

- **Raw streaming** -- accumulate deltas, parse once at block completion (not O(n^2) partial parsing).
- **API preconnection** -- warm the TCP+TLS handshake during initialization.
- **Prompt cache structure** -- stable content first, volatile content last.
- **Global cache scope** -- share cache entries across users of the same Claude Code version.

The result: reduced latency and 90% discount on cached input tokens.

## Tool Execution Optimization

Chapters 6 and 7 described the tool system. The optimizations are:

- **Concurrent execution** -- independent tools run in parallel, not sequentially.
- **Streaming executor** -- start tools while the API response is still streaming.
- **Result budgeting** -- persist oversized results to disk, not conversation history.
- **File cache** -- LRU cache with staleness checking avoids redundant disk reads.

The result: 5x speedup on multi-tool turns, reduced token usage.

## Memory Optimization

Long-running sessions can accumulate large state. The optimizations are:

- **Bounded caches** -- LRU eviction prevents unbounded growth.
- **Context compaction** -- summarize old conversation history when approaching context limits.
- **Lazy loading** -- sub-agent transcripts loaded on demand, not at startup.

The result: stable memory usage even in multi-hour sessions.

## Profiling and Measurement

All optimizations are data-driven. Claude Code includes extensive profiling:

- **Startup checkpoints** -- 50+ timing points from process start to interactive prompt.
- **API telemetry** -- token usage, latency, cache hit rates.
- **Rendering metrics** -- frame timing, cell update counts.
- **Tool execution timing** -- per-tool execution time and result size.

The data is sampled (100% for internal users, 0.5% for external) and sent to telemetry. It drives optimization decisions and regression detection.

## Apply This: Building Performant Systems

**Profile first, optimize second.** Do not guess where the bottlenecks are. Measure. The 50+ startup checkpoints revealed that keychain reads were sequential -- the fix was obvious once the data was visible.

**Parallelize by default.** CPU and I/O can overlap. Network and disk can overlap. Look for opportunities to do multiple things at once.

**Cache intelligently.** Caching is not free -- it adds complexity and can serve stale data. Cache at the right level (file contents, not file metadata). Check staleness. Bound the cache size.

**Measure in production.** Local profiling is useful, but real users have different patterns. Telemetry with sampling gives you production data without overwhelming the pipeline.

# Chapter 18: What We Learned

## Architecture Principles

After eighteen chapters examining every subsystem of Claude Code, what have we learned? What principles emerge from the implementation details?

**Explicit over implicit.** The agent loop is a single async generator, not a distributed callback system. Every state transition is visible in the control flow. Every terminal state is encoded in a discriminated union. When something goes wrong, there is one place to look.

**Fail closed.** Tools default to serial execution until proven safe for concurrency. Permissions default to prompting until explicitly allowed. New features are gated behind feature flags that default to off. The system errs on the side of safety.

**Measure everything.** Startup has 50+ profiling checkpoints. API calls track token usage and cache hit rates. The rendering engine reports frame timing. Performance optimizations are driven by data, not intuition.

## Design Patterns

Several patterns recur throughout the codebase:

**Generators for streaming.** The `query()` function, `runAgent()`, and `runTool()` are all async generators. This enables streaming output, cancellation, and interleaved execution.

**Discriminated unions for state.** Terminal states, tool results, and message types are all discriminated unions. The type system enforces that all cases are handled.

**Context passing for isolation.** Sub-agents receive a carefully constructed context that isolates them from the parent while allowing necessary coordination.

**Hooks for extensibility.** The hook system lets users customize behavior without modifying core code. PreToolUse, PostToolUse, Stop, and other hooks provide extension points at critical lifecycle events.

## Trade-offs

Every design involves trade-offs. Claude Code makes several explicit choices:

**Terminal vs. GUI.** The terminal is limiting -- no rich graphics, no mouse interactions. But it is universal, scriptable, and lightweight. The trade-off favors ubiquity over richness.

**Local vs. cloud.** Local execution is private and fast but limited to one machine. Remote execution extends reach but adds complexity and security concerns. The trade-off favors local-by-default with remote as extension.

**Built-in vs. external tools.** Built-in tools are reliable and fast but limited in number. MCP servers extend capabilities but introduce external dependencies. The trade-off favors a solid core with extensible periphery.

## What Comes Next

Claude Code is not finished. The architecture described in this book is a snapshot of a system in evolution. Future directions include:

- **Smarter caching.** Beyond prompt caching to semantic caching -- recognizing when a similar question has been asked before.
- **Richer UI.** Exploring what is possible within terminal constraints -- better visualizations, more interactive elements.
- **Deeper integration.** Tighter integration with development workflows -- IDE plugins, CI/CD hooks, automated testing.
- **Broader model support.** Extending beyond Anthropic's models to other providers while maintaining the same user experience.

The principles will remain. Explicit state machines. Fail-closed defaults. Data-driven optimization. These are not tied to any specific implementation; they are timeless software engineering practices.

## Final Thoughts

Building an agentic system is not about the model. The model is a component -- a powerful one, but still just a component. The system is what makes the model useful: the state management, the tool orchestration, the permission system, the UI, the extensibility hooks.

Claude Code demonstrates that a well-architected system can make a capable model feel magical. The model provides the intelligence. The system provides the structure that lets that intelligence operate effectively in the real world.

The source code is the ultimate reference. This book has explained the why and how. The code shows the what. Read both. Build your own. The future of software is agentic, and the architecture matters.